AI Accelerator Program

Analytics Vidhya

Popular AI Agents Frameworks

crewai

Agno

AG

n8n

No-Code / Low-Code Tool

Analytics Vidhya

n8n

n8n is currently one of the most powerful no-code tools for building AI agents integrated with real-world systems. n8n works as a node-based workflow automation system where each node performs an operation:

- Trigger
- API Call
- LLM Execution
- Conditional logic
- Database access
- Tool execution

With AI, it becomes an agent orchestration engine.

It supports

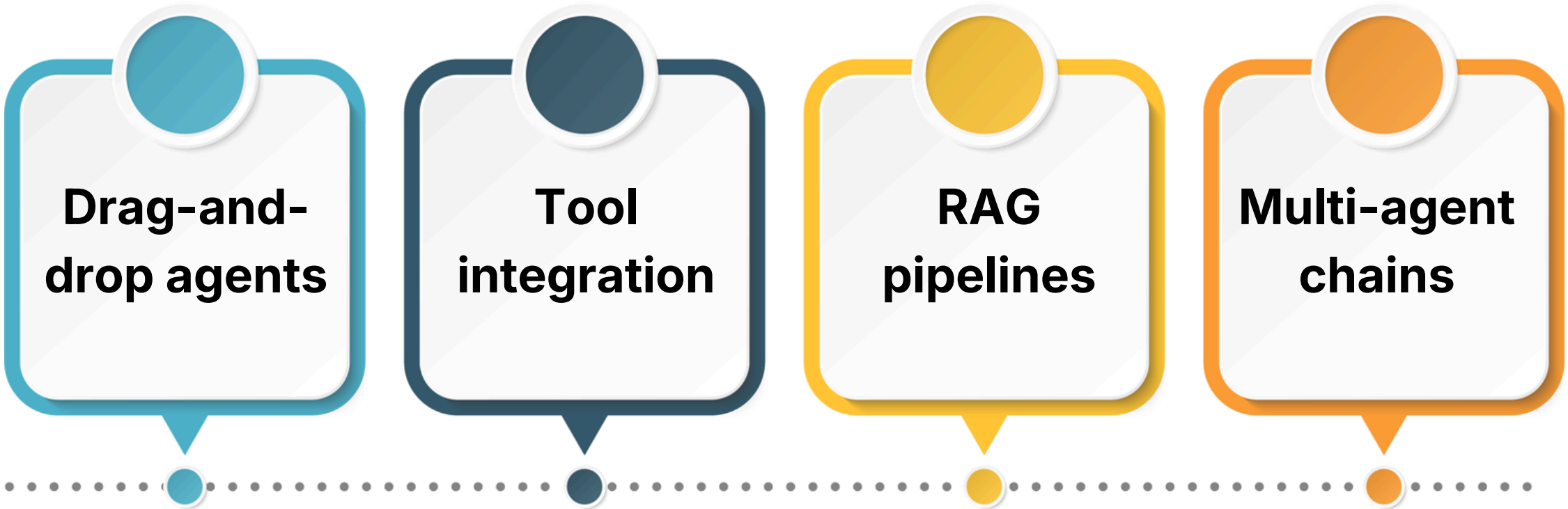
- Tool calling
- Memory via databases
- Vector DB integration
- Multi-step reasoning workflows

Example Flow:

User message → LLM Node → Tool Decision → Execute API → Return Result

Flowise

Flowise is an open-source visual agent orchestration framework designed to build LLM applications and AI agents using a drag-and-drop interface. It provides modular building blocks that allow developers to create everything from simple chatbots to fully autonomous multi-agent systems without writing backend orchestration code. It allows:



Internally, it runs LangChain.

Best use case: Quick prototyping LangChain agents visually

No-Code / Low-Code Tool

Analytics Vidhya

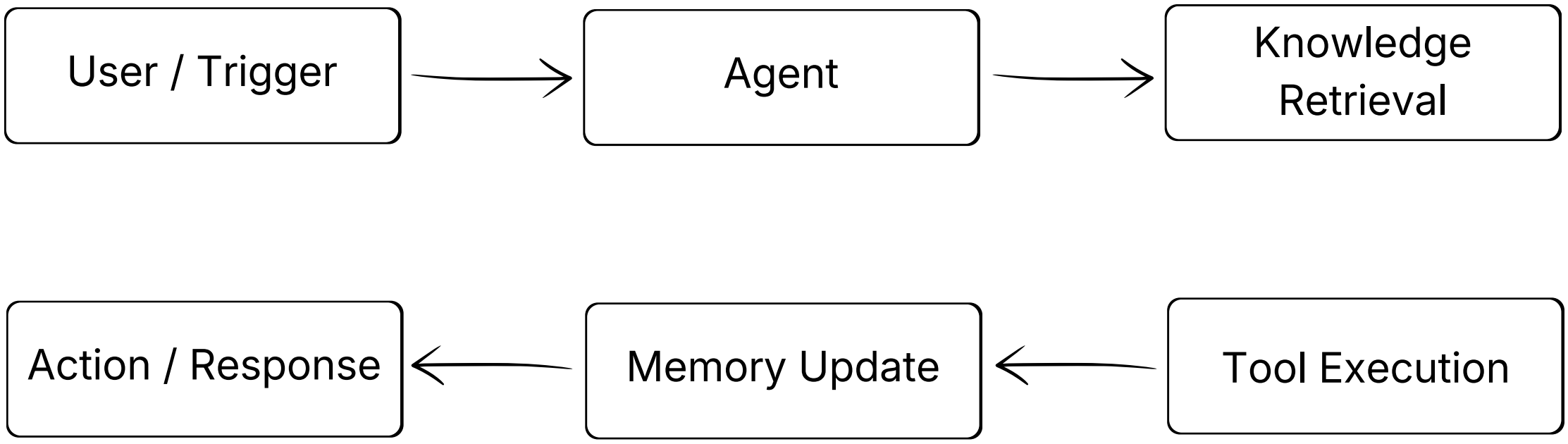
Relevance AI

Relevance AI is an enterprise-focused agent platform designed to build, deploy, and manage business-ready AI agents with strong capabilities in knowledge integration, workflow automation, and operational decision-making.

Unlike general agent frameworks, Relevance AI focuses specifically on business operations agents such as sales agents, support agents, and research agents.

Its core philosophy is: Agents should perform real business functions, not just conversations.

Relevance AI provides a complete agent stack consisting of:



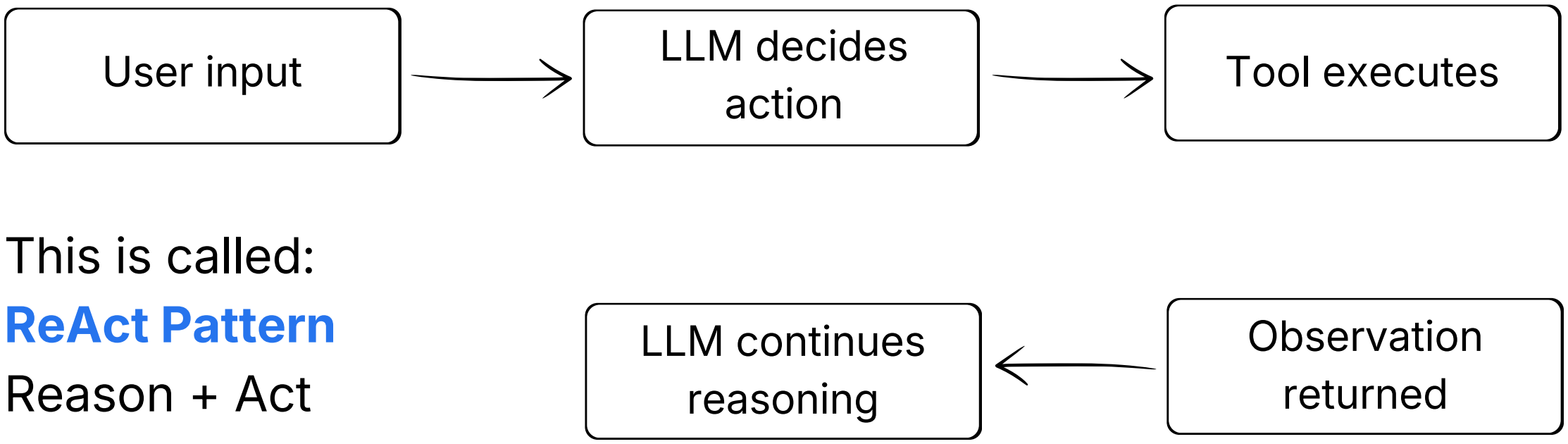
Code-Based Frameworks

Analytics Vidhya

🦜🔗 LangChain

LangChain is the most widely used agent framework.
It provides abstractions for: Models, Tools, Memory, Chains, Agents, Retrievers, Vector stores. This is how LangChain works internally:

Agent Loop:



This is called:
ReAct Pattern
Reason + Act

```
from langchain.agents import create_agent

agent = create_agent(
    model="openai:gpt-5.1 mini",
    tools=[search_tool],
)
```

Strengths

- Huge ecosystem
- Supports all LLM providers
- Flexible
- Production tested

Weaknesses

- Too abstract sometimes
- Complex debugging
- Performance overhead

Code-Based Frameworks

Analytics Vidhya

LangGraph

LangGraph is the next evolution of LangChain. It solves the biggest problem:

➡ Agent control flow

➡ LangChain is linear.

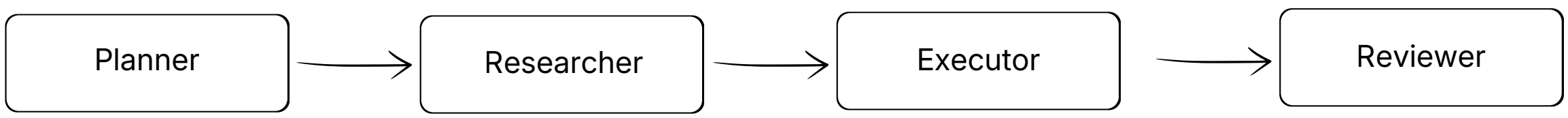
➡ LangGraph is graph-based.

Instead of:

Chain → Chain → Chain

You define: **Graph nodes**

Each node is: **Agent**, **Tool**, **Function**, **LLM**




```
graph.add_node("planner", planner_fn)
graph.add_node("executor", executor_fn)

graph.add_edge("planner", "executor")
```

LangGraph is currently the most powerful framework for production agents.

Code-Based Frameworks

Analytics Vidhya

crewai

CrewAI focuses on role-based multi-agent systems.
It introduces three core abstractions:

- Agents
- Tasks
- Crew

```
researcher = Agent(
 role="Researcher",
 goal="Find information"
)
```

Crew executes tasks collaboratively.

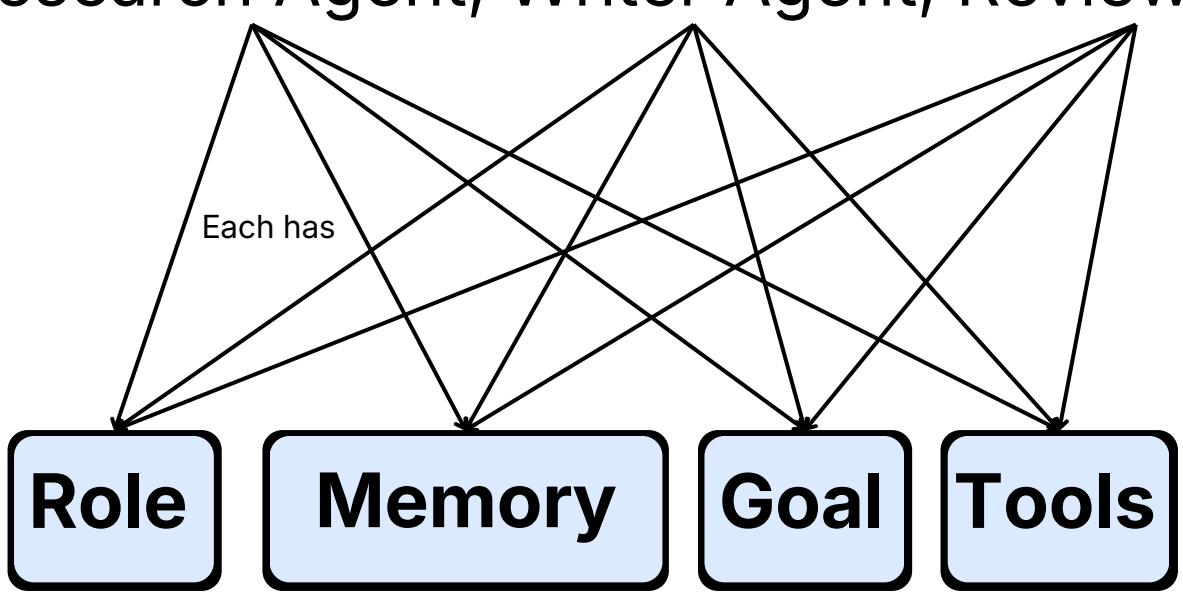
Strengths

- Very intuitive multi-agent design

Weaknesses

- Less flexible than LangGraph

Examples: Research Agent, Writer Agent, Reviewer Agent

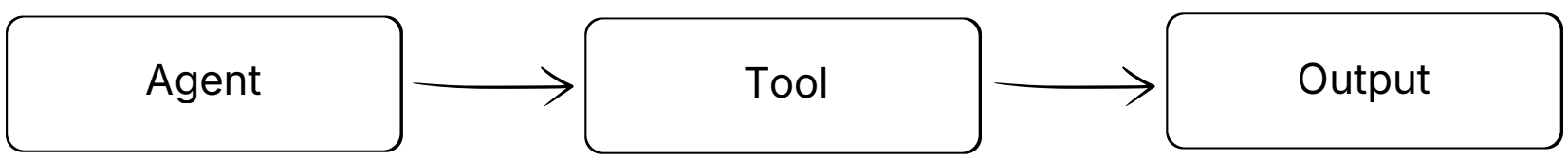


Code-Based Frameworks

AutoGen

AutoGen is an open-source agent framework developed by Microsoft for building conversational, collaborative, and autonomous multi-agent systems. It is specifically designed to enable agents to communicate with each other, coordinate tasks, and solve complex problems through dialogue.

AutoGen treats agents as independent entities that collaborate via conversation. Instead of:



This creates collaborative intelligence.

```
assistant = AssistantAgent()
user_proxy = UserProxyAgent()
```

Code-Based Frameworks

Analytics Vidhya

Agno

Agno is a modern lightweight agent framework focused on simplicity and performance.

It is designed to fix complexity issues in LangChain. Some of the key features are mentioned below:

- Minimal abstraction
- Fast
- Simple agent creation
- Built-in tool usage
- Memory support
- Multi-agent support

```
agent = Agent(
 model="gpt-4",
 tools=[search_tool]
)
```

LlamaIndex

LlamaIndex is a data-centric agent framework designed to connect LLM agents with structured and unstructured data.
Originally built for RAG, it has evolved into a full agent framework.
Where LangChain focused on tool orchestration, LlamaIndex focused on knowledge orchestration. Now it does both.

LlamaIndex is used to connect large language models with external data like PDFs, databases, or documents so the model can answer questions based on real, relevant information instead of guessing. It handles key steps such as organizing data, creating embeddings, storing them in vector databases, and retrieving the right context during a query. This makes it much easier to build accurate applications like document chatbots, knowledge assistants, and AI copilots without implementing the entire retrieval pipeline from scratch.

LlamaIndex treats data as the primary foundation of agents.
Its architecture is built around:

Data → **Index** → **Retriever** → **Agent** → **Tool execution**

Instead of just calling tools, agents query knowledge intelligently